

Convenciones de denominación

- **Pascal case:**
 - Inicial en mayúsculas, demás caracteres en minúsculas.
 - Palabras compuestas se separan utilizando la inicial de cada palabra en Mayúsculas.
- **Camel case:**
 - Toda la palabra en minúsculas.
 - Palabras compuestas se separan utilizando la inicial de cada palabra en Mayúsculas.
- **Reglas generales:**
 - Utilizar nombres nemotécnicos (que representen el contenido y / o función que cumplen).
 - Sustantivos para las variables / properties, expresiones verbales para los métodos.
 - Evitar las abreviaturas.
 - Utilizar acrónimos solo cuando se reconozcan fácilmente (ej.: Rut, Iva)

ELEMENTO	CONVENCIÓN	EJEMPLOS
Clases, structs, enums	PascalCase	PedidoUrgente, CUAltaUsuario
Métodos y propiedades	PascalCase	CalcularTotal(), ValidarNombre()
Variables locales y parámetros	camelCase	nombreUsuario, numeroHabitacion
Interfaces	PascalCase + prefijo I	IRepositorio, IValidable
Atributos privados	camelCase + guion bajo inicial	_id, _nombreCliente
Atributos privados estáticos	camel case + prefijo "s_"	s_ultId, s_maxProductos
Atributos públicos de solo lectura	PascalCase	ConnectionString

Comentarios

- Escribir los comentarios en una línea aparte, no al final de la sentencia.
- No armar recuadros con asteriscos
- Incluir en las clases, interfaces, y métodos públicos los comentarios XML que permitan generar la documentación.

Convenciones de código en C#

Los comentarios de los métodos y clases deben ser de la forma:

<pre>/// <summary> /// /// /// </summary> public class Factura { }</pre>	<pre>/// <summary> /// ... /// </summary> /// <returns>...</returns> public decimal CalcularArea() { }</pre>
---	---

Controlador MVC

Cada controlador tiene la responsabilidad de controlar una clase del modelo y el nombre va en plural, sufijado con “Controller”:

Ej.: PropiedadesController

Nota: esta es una regla general, que puede no ser siempre aplicable. Por ejemplo, supongamos que tenemos la clase Proyecto y la clase Tarea, cada proyecto contiene varias tareas y quiero que al visualizar el listado de proyectos pueda clicar sobre uno en particular y ver sus tareas. En este caso desde ProyectosController obtendría las tareas correspondientes.

Foreach

Para recorrer listas usando el foreach no utilizar *var* para declarar el iterador.

```
foreach (Propiedad item in _propiedades)
{
    ...
}
```

Tareas pendientes: TODO

Cuando no esté terminado un método o falta implementarlo ponemos comentario con la palabra ToDo

Para ver todos los pendientes podemos usar: ctrl + \ + T (ctrl + º + T) o en el menú Ver > Lista de Tareas.

Ajustar código

Ctrl + K + D permite formatear el código del documento que estamos utilizando.

Después de cada método solo debe ir un espacio en blanco

Cada vez que definamos un método o una clase debemos pensar como hacerla y es bueno saber que vamos a usar los principios SOLID

- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

La idea es que cada método debe tener una única responsabilidad

En este caso tenemos un método que se llama nombre y lo que hace es solicitar el nombre y mostrarlo, realizando 2 acciones. Para que haga una única responsabilidad debemos hacer 2 métodos

```
1 referencia
public static void Nombre()
{
    string nombre;
    Console.WriteLine("Ingresa tu nombre");
    nombre = Console.ReadLine();
    Console.WriteLine(nombre);
}
```

Realizando 2 sencillos métodos logramos resolver

```
static void Main(string[] args)
{
    string nombre = PedirTexto("Ingrese su nombre");
    MostrarNombre(nombre);
}

// Metodos para mostrar Datos
1 referencia
public static void MostrarNombre(string nombre)
{
    Console.WriteLine(nombre);
}

// Metodos para solicitar datos
1 referencia
public static string PedirTexto(string mensaje)
{
    Console.WriteLine(mensaje);
    return Console.ReadLine();
}
```

Escribir inline cuando no se usa la variable

En este caso vemos que la variable nombre solo es usada una vez para guardar el dato lo que nos indica que esa variable no es necesaria

```
0 referencias
static void Main(string[] args)
{
    MostrarNombre(PedirTexto("Ingrese su nombre"));
}
}
```

Migraciones

- **Utilice nombres de migración significativos** : no nombre las migraciones con fechas o descripciones genéricas. Utilice nombres claros y descriptivos que indiquen el propósito de la migración. Buenos ejemplos: **AddProductsTable**, **RenameDescriptionToShortDescription**.
- **Mantenga las migraciones pequeñas y centradas** : evite crear migraciones masivas que contengan múltiples cambios no relacionados. Las migraciones más pequeñas son más fáciles de revisar, probar y solucionar si algo sale mal. Procure realizar una migración por función o cambio lógico.

Git

Este documento describe las buenas prácticas y el flujo de trabajo recomendado para equipos que desarrollan proyectos obligatorios utilizando **Git y GitHub**.

1. Crear el repositorio

- Crear un **repositorio privado** en GitHub.
- Incluir los archivos:
 - README.md

Debe contener información clara y útil sobre el proyecto, por ejemplo:

- **Nombre del proyecto**
- **Descripción** breve del objetivo y alcance
- **Diagrama general del proyecto** (puede ser un diagrama de clases, de paquetes o de flujo, según corresponda)
- **Estructura de paquetes / carpetas**
- **Instrucciones para ejecutar el proyecto**
- **Dependencias** necesarias (librerías, frameworks, versiones de .NET u otros)

- **Autores y contactos** de los integrantes del equipo

- .gitignore configurado para **Visual Studio**

2 Clonar y actualizar el repositorio

- Cada integrante debe clonar el repositorio en su máquina local antes de comenzar a trabajar.
- Antes de empezar cualquier nueva tarea o crear una rama, se debe actualizar la rama main local con los cambios que existan en el repositorio remoto.
- Esto asegura que todos trabajen siempre con la versión más reciente del proyecto y ayuda a evitar conflictos innecesarios.

3 Ramas y commits

- Cada integrante debe **trabajar en su propia rama** para implementar funcionalidades.
- Se recomienda **una rama por caso de uso o funcionalidad**, para mantener los cambios organizados y facilitar la integración al proyecto principal.
- **Nombre de la rama:** reflejar claramente el caso de uso o funcionalidad.
- **Mensajes de commit:** describir de manera clara los cambios realizados.
- **Prefijos recomendados para commits:**

Prefijo	Significado
feat:	Nueva funcionalidad
fix:	Corrección de errores
chore:	Tareas de mantenimiento sin impacto funcional
docs:	Cambios en la documentación
style:	Cambios de estilo sin afectar el código
refactor:	Cambios en el código sin agregar o corregir funcionalidad
test:	Añadir o modificar pruebas
perf:	Mejoras de rendimiento

Prefijo	Significado
build:	Cambios en el sistema de build o dependencias
ci:	Cambios en la configuración de CI/CD
revert:	Reversión de un commit anterior

4 Pull Requests y revisión de código

- Los pull requests **no deben ser aceptados por la misma persona que los creó**. Deben ser revisados por el compañero de equipo.
- Al crear un PR, incluir:
 - Qué problema resuelve
 - Qué cambios introduce
 - Cómo probarlo
 - Aspectos a revisar con atención
- Antes de solicitar revisión:
 - Ejecutar tests
 - Verificar que el código compile correctamente
 - Usar herramientas de lint/formato si están configuradas

5. Resolución de conflictos

- En caso de conflictos, **ambos integrantes deben resolverlos conjuntamente**, tomando decisiones de manera coordinada.

Sección: Comandos Git esenciales

Esta sección reúne los comandos más utilizados durante el desarrollo del obligatorio, organizados por función:

1. Repositorio

- **Clonar repositorio:**
git clone <URL-del-repositorio>
- **Ver estado del repositorio:**
git status

2. Actualizar la rama principal

- **Cambiar a main:**
`git checkout main`
- **Actualizar main con cambios remotos:**
`git pull origin main`

3. Ramas

- **Crear nueva rama:**
`git checkout -b <nombre-rama>`
- **Cambiar de rama:**
`git checkout <nombre-rama>`
- **Listar ramas locales:**
`git branch`
- **Listar ramas remotas:**
`git branch -r`

4. Commit y cambios

- **Agregar cambios al área de staging:**
`git add <archivo>` o `git add .`
- **Hacer commit:**
`git commit -m "prefijo: descripción del cambio"`
- **Ver historial de commits:**
`git log`

5. Push y Pull

- **Enviar cambios a remoto:**
`git push origin <nombre-rama>`
- **Actualizar rama local desde remoto:**
`git pull origin <nombre-rama>`

6. Merge

- **Combinar rama a main:**
`git checkout main`
`git merge <nombre-rama>`

7. Pull Requests (opcional)

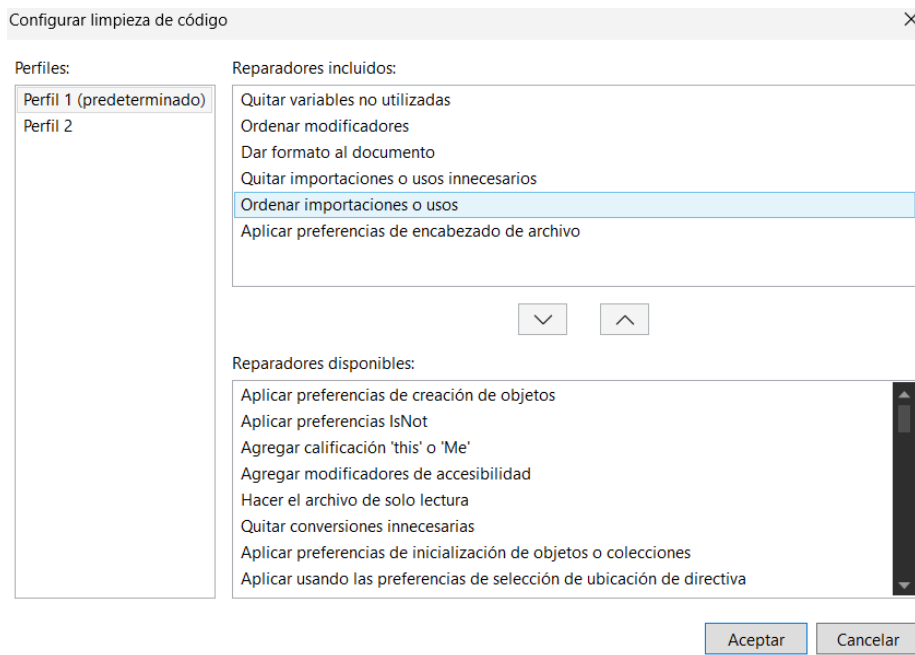
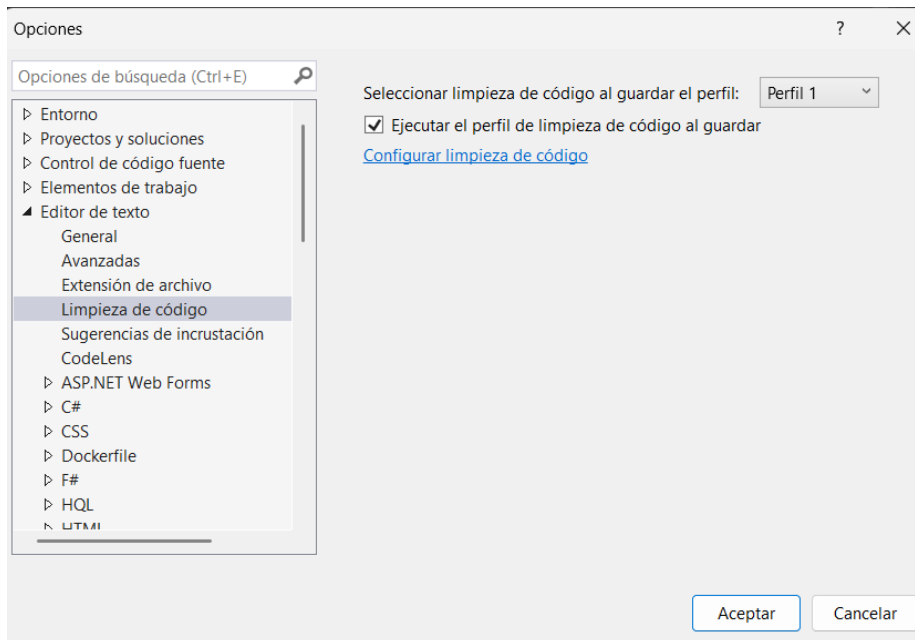
- Crear pull requests desde GitHub para integrar ramas al main.

8. Resolución de conflictos

- Editar archivos conflictivos, luego:
git add <archivo>
git commit -m "Resolviendo conflicto"

Guardar documento

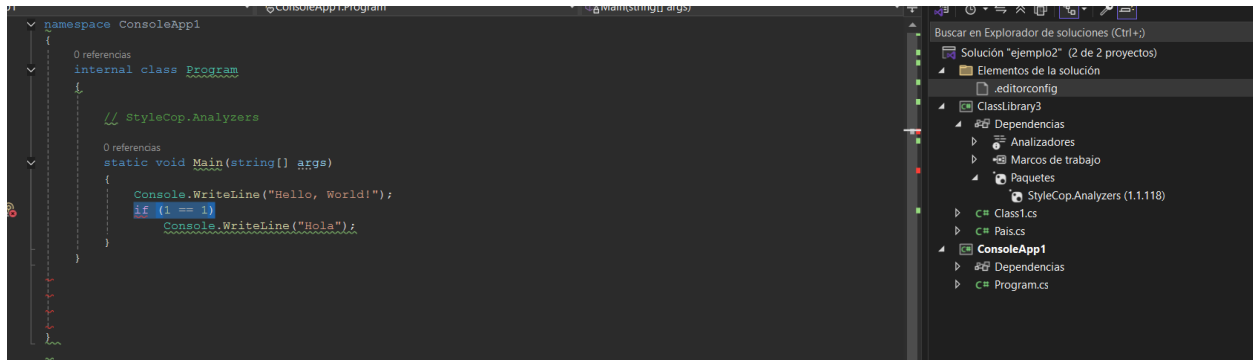
Podemos realizar ajustes a nuestro editor para que al guardar el documento haga ciertas acciones para mejorar el código. En herramienta / opciones



Visual Studio

Instalar el paquete StyleCop.Analyzers en todos los proyectos

Luego crear el archivo de reglas en la solución .editorconfig



Este archivo define reglas de estilo para todos los archivos C# del proyecto

Es la raíz de configuración, no se buscan otros .editorconfig en carpetas superiores

root = true

Estas reglas se aplican a todos los archivos con extensión .cs

[*.cs]

#####

Reglas generales de formato

#####

Usa espacios (no tabs) para indentación

indent_style = space

La indentación será de 4 espacios

indent_size = 4

Convenciones de código en C#

Agrega una nueva línea al final de cada archivo

insert_final_newline = true

Elimina espacios en blanco al final de las líneas

trim_trailing_whitespace = true

#####

Preferencias de estilo específicas de C#

#####

Obliga al uso de llaves en bloques como if, even si es una sola línea

csharp_prefer_braces = true:error

Prefiere usar propiedades automáticas (auto-properties)

dotnet_style_prefer_auto_properties = true:error

#####

Reglas StyleCop – Organización y nombres

#####

SA1300: Clases, métodos, namespaces → PascalCase obligatorio

dotnet_diagnostic.SA1300.severity = error

SA1302: Interfaces deben comenzar con "I" (ej: IValidable)

dotnet_diagnostic.SA1302.severity = warning

SA1309: Campos privados deben comenzar con "_" (ej: _nombre)

dotnet_diagnostic.SA1309.severity = warning

Convenciones de código en C#

SA1210: Usings deben estar ordenados alfabéticamente

dotnet_diagnostic.SA1210.severity = warning

SA1200: Usings dentro del namespace (sugerido agregar)

dotnet_diagnostic.SA1200.severity = warning

#####

Reglas StyleCop – Comentarios y documentación

#####

SA1600: Comentarios XML requeridos en clases y métodos públicos

dotnet_diagnostic.SA1600.severity = warning

#####

Reglas StyleCop – Espaciado y sangría

#####

SA1000: Espacio requerido entre keyword y paréntesis (if (), while (), etc.)

dotnet_diagnostic.SA1000.severity = error

SA1001: Espaciado correcto en signos de puntuación (ej: , ; [])

dotnet_diagnostic.SA1001.severity = error

SA1025: Evita espacios al final de las líneas

dotnet_diagnostic.SA1025.severity = error

SA1027: Espaciado incorrecto en operadores (como +=, =, etc.)

dotnet_diagnostic.SA1027.severity = error

Convenciones de código en C#

Espacio antes y después de operadores binarios (ej: a + b)

csharp_space_around_binary_operators = before_and_after

SA1500: Bloques deben estar correctamente indentados

dotnet_diagnostic.SA1500.severity = error

SA1507: Debe haber una línea en blanco después de cerrar un bloque

dotnet_diagnostic.SA1507.severity = error

Indentación especial para etiquetas como case/default

csharp_indent_labels = one_less_than_current

Más información:

<https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>

<https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>